KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

FINAL PROJECT REPORT

MARKET PRICE AND CROP PLANNING HELPER

An AI-Assisted Decision Support System for Crop Suitability, Market Insights and Farm Planning

Submitted by
MARRI VENKATA KASI VARDHAN
MEKALA SAI GANESH
YETTAPU GANESH KUMAR REDDY
PARTIPATI DHRUVA KUMAR

Project Team: ORBIT 431
Department: CSE (Artificial Intelligence and Machine Learning)
Academic Year: 2026–2027
Project Guide: [Enter Guide Name]

CERTIFICATE

This is to certify that the project entitled “Market Price and Crop Planning Helper” is a bona fide project work carried out by the students listed below under the guidance and supervision of the undersigned, in partial fulfilment of the requirements of the applicable academic/training programme.

Project Team: ORBIT 431
MARRI VENKATA KASI VARDHAN
MEKALA SAI GANESH
YETTAPU GANESH KUMAR REDDY
PARTIPATI DHRUVA KUMAR

Guide Name and Signature: _____

Head of Department: _____

Date: _____ Place: _____

DECLARATION

We declare that the project report entitled “Market Price and Crop Planning Helper” is a record of the work carried out by our team. Sources, tools and external services used in the project are acknowledged in the references. We have reviewed the report and will verify all institutional and technical details before final submission.

Student signatures:
MARRI VENKATA KASI VARDHAN _____
MEKALA SAI GANESH _____
YETTAPU GANESH KUMAR REDDY _____
PARTIPATI DHRUVA KUMAR _____

ACKNOWLEDGEMENT

We express our sincere gratitude to Kalasalingam Academy of Research and Education, our department, faculty members and project review experts for their support. We thank the authors and organizations whose documentation, agricultural publications and open-source software helped us implement the system. We also acknowledge the public data services and developer platforms used during implementation and thank our teammates for their collaboration.

TABLE OF CONTENTS

01. ABSTRACT

02. CHAPTER 1 – INTRODUCTION

03. CHAPTER 2 – PROBLEM DEFINITION AND OBJECTIVES

04. CHAPTER 3 – EXISTING AND PROPOSED SYSTEM

05. CHAPTER 4 – REQUIREMENTS AND TECHNOLOGY STACK

06. CHAPTER 5 – SYSTEM ARCHITECTURE AND WORKFLOW

07. CHAPTER 6 – METHODOLOGY AND IMPLEMENTATION

08. CHAPTER 7 – API AND USER INTERFACE

09. CHAPTER 8 – TESTING AND RESULTS

10. CHAPTER 9 – DEPLOYMENT

11. CHAPTER 10 – CONCLUSION AND FUTURE ENHANCEMENTS

12. REFERENCES

13. APPENDIX – DETAILS TO CONFIRM BEFORE SUBMISSION

ABSTRACT

The Market Price and Crop Planning Helper is an AI-assisted agricultural decision-support prototype. It brings farm-input capture, crop suitability assessment, financial estimation, market-price access and evidence-grounded agricultural knowledge retrieval into one application. The frontend uses React and Vite, while the backend uses Python and FastAPI. LangGraph coordinates the planning workflow. Deterministic modules score crop suitability and calculate investment and return indicators. A Retrieval-Augmented Generation (RAG) pipeline retrieves relevant passages from a FAISS index, with query embeddings generated using a MiniLM ONNX model through ONNX Runtime. Groq is configured as the language-model provider for conversational explanations.

The system is intended to help users explore crop-planning scenarios. It does not guarantee yield, profit or market prices and does not replace local agricultural expertise. Recommendations depend on input quality, benchmark assumptions and the freshness and regional relevance of external data.

CHAPTER 1 – INTRODUCTION

1.1 Background

Farmers must consider soil, water availability, season, temperature, rainfall, land area, cultivation cost, expected yield and market prices when selecting crops. These details are often distributed across multiple sources. A unified decision-support tool can help organize the information and make comparisons easier.

1.2 Purpose and Scope

The project provides a web-based workflow for entering farm details, evaluating crop options, estimating financial outcomes and retrieving supporting agricultural information. Its scope includes crop suitability, financial estimation, public mandi-price integration, RAG-based knowledge retrieval and a conversational explanation layer.

1.3 Intended Users

The intended users are farmers, students and project evaluators exploring crop-planning scenarios. Outputs should be checked against local soil tests, current agricultural advisories and actual market conditions.

CHAPTER 2 – PROBLEM DEFINITION AND OBJECTIVES

2.1 Problem Statement

Farm-planning information is fragmented and farmers may not have a consistent way to compare crop suitability, investment requirements and possible returns. Generic recommendations can be misleading when they ignore local constraints or current market conditions.

2.2 Objectives

• Provide a responsive interface for farm details.

• Evaluate crop suitability using transparent weighted criteria and hard constraints.

• Estimate cultivation cost, yield, revenue, net profit, ROI, break-even and cost per kilogram.

• Retrieve relevant agricultural knowledge from an indexed knowledge base.

• Integrate mandi-price information through a configurable public API.

• Coordinate the planning workflow with LangGraph and expose it through FastAPI.

• Validate inputs and test important API error cases.

• Deploy the frontend and backend in a maintainable configuration.

CHAPTER 3 – EXISTING AND PROPOSED SYSTEM

3.1 Existing Approach

In a manual approach, farmers may consult traders, government portals, agricultural officers, weather sources and personal experience. Calculations are often performed separately, making comparisons time-consuming and assumptions inconsistent.

3.2 Proposed System

The proposed application combines farm-input capture, suitability scoring, financial calculations, market-data access and knowledge retrieval. Rule-based components handle structured calculations. RAG retrieves supporting passages, and the

language model can help explain results.

3.3 Advantages and Limitations

• Centralized crop-planning workflow.

• Transparent scoring criteria and financial indicators.

• Retrieval of supporting agricultural passages.

• Modular backend and lighter ONNX-based embedding inference.

• Results depend on the accuracy and freshness of source data.

• Estimated financial outcomes are not guaranteed and must be checked locally.

CHAPTER 4 – REQUIREMENTS AND TECHNOLOGY STACK

4.1 Functional Requirements

• Accept location, soil, water, season and land area.

• Validate required fields and supported values.

• Evaluate crop suitability and financial indicators.

• Retrieve relevant knowledge-base passages.

• Display the planning response in the frontend.

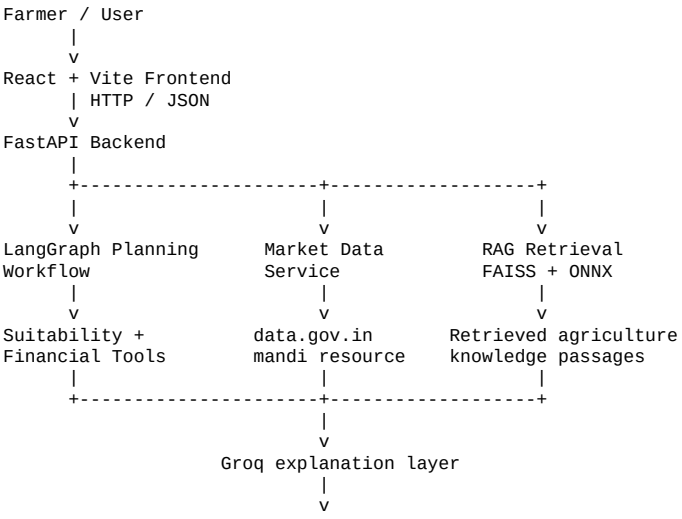
• Request configured mandi-price records.

4.2 Technology Stack

Layer	Technology	Purpose
Frontend	React, Vite, Tailwind CSS	User interface
Backend	Python, FastAPI, Uvicorn	REST API
Workflow	LangGraph	Multi-step orchestration
LLM provider	Groq / LangChain integration	Conversational explanations
Vector search	FAISS	Similarity retrieval
Embeddings	MiniLM ONNX, ONNX Runtime, Tokenizers	Query embeddings
Market data	data.gov.in	Mandi-price records
Validation	Pydantic, FastAPI TestClient	Request validation and testing
Hosting	Render	Deployment target

CHAPTER 5 – SYSTEM ARCHITECTURE AND WORKFLOW

5.1 High-Level Architecture



```
Structured API response
```

5.2 Workflow

1. Collect farm location, soil, water, season and land area.
2. Validate request fields and supported values.
3. Load candidate crop profiles and benchmark values.
4. Apply hard soil and season constraints.
5. Calculate weighted suitability scores.
6. Estimate investment, production, revenue, profit and ROI.
7. Retrieve relevant agricultural passages through FAISS.
8. Use the configured LLM to explain results with retrieved context.
9. Return structured output to the frontend.

5.3 Architecture Principle

The design separates deterministic scoring and financial arithmetic from generative explanations. This reduces the risk of relying on a language model for calculations that should be reproducible.

CHAPTER 6 – METHODOLOGY AND IMPLEMENTATION

6.1 Crop Data

The crop-data module stores crop benchmark information and provides functions to retrieve an individual crop profile and list available crops. During development, the crop data set was expanded from four to ten crops, using ICAR/ANGRAU-oriented benchmark references. Exact values and their regional applicability should be verified against the publications actually used by the team.

6.2 Crop Suitability Scoring

The implementation uses five weighted criteria: soil 25%, water 20%, season 15%, temperature 20% and rainfall 20%. A general score is calculated as 0.25S + 0.20W + 0.15Se + 0.20T + 0.20R, where each component is normalized to a common scale. Hard soil and season constraints are applied separately so that a high score in other categories cannot compensate for a fundamentally incompatible condition.

Criterion	Weight
Soil suitability	25%
Water availability	20%
Season suitability	15%
Temperature suitability	20%
Rainfall suitability	20%
Total	100%

6.3 Financial Estimation

The financial tool estimates total cultivation cost, expected yield, gross revenue, net profit, ROI, break-even and cost per kilogram. Core formulas include: Gross Revenue = Expected Price × Expected Production; Net Profit = Gross Revenue – Total Cost; ROI (%) = (Net Profit ÷ Total Cost) × 100; Cost per kg = Total Cost ÷ Production in kg; Break-even quantity = Total Cost ÷ Selling Price. Each division requires a positive denominator. Conservative, expected and optimistic scenarios communicate uncertainty rather than certainty.

6.4 Recommendation Ranking

Recommendations should prioritize agronomic suitability, then affordability and expected financial outcome. Ranking only by estimated profit could select an unsuitable crop or one requiring more investment than the farmer can afford. Budget validation is used when a budget is supplied.

6.5 Market-Price Integration

The market service is configured to request mandi-price information from a data.gov.in resource. API keys and resource configuration are supplied through environment variables. Records should be interpreted with commodity, market, variety, grade and date. A mandi quotation is not necessarily the farmer's final realized price after transport, commission, grading or other charges.

6.6 RAG Retrieval

Agricultural documents are prepared as text chunks, embedded as vectors and stored in a FAISS index with metadata. At query time, the question is tokenized, converted into an embedding and compared with indexed vectors. The most relevant passages are returned as context for the answer. The project index contains 34 vectors of dimension 384 with corresponding metadata.

6.7 ONNX Runtime Optimization

The earlier PyTorch sentence-transformer inference path contributed to memory pressure on the free hosting instance. The query embedding path was changed to a MiniLM ONNX model using ONNX Runtime and Hugging Face Tokenizers. The model file is approximately 86.21 MB. The inference session is configured conservatively with one intra-op thread and sequential execution. Local validation showed an embedding shape of (1, 384), finite float32 values and normalized magnitude. Retrieval tests returned relevant indexed passages.

6.8 LangGraph and Groq

LangGraph coordinates the planning workflow and its tools. Groq is configured through the LangChain integration as the language-model provider. The model layer explains computed results and retrieved evidence; core suitability scoring and financial arithmetic remain in application code. API keys must be stored as secret deployment environment variables and must not be committed to the repository.

CHAPTER 7 – API AND USER INTERFACE

7.1 API Endpoints

Endpoint	Method	Purpose
/health	GET	Health check
/	GET	Serves frontend build when available
/openapi.json	GET	OpenAPI schema
/api/plan	POST	Generate crop-planning response
/api/market/collect	POST	Trigger market-data collection
/api/market/analytics/{crop}	GET	Retrieve crop market analytics

7.2 Request Validation

The planning request requires location, soil, water, season and land area. Optional fields include a farmer question and budget. Pydantic validation rejects missing required fields and unsupported soil or water values before invoking the planning workflow.

7.3 Frontend and Configuration

The React frontend collects farm details and displays the planning result. Vite creates the production bundle. The frontend uses a configurable API base URL, allowing same-origin requests when frontend and backend are hosted together. Secret values belong in server-side environment configuration.

CHAPTER 8 – TESTING AND RESULTS

8.1 Testing Approach

Development checks included Python compilation, dependency consistency, RAG retrieval, frontend production build and API request validation.

8.2 Recorded Development Results

Test	Recorded result
Python compilation / compileall	Passed
Local dependency consistency check	Passed

Test	Recorded result
FAISS index	34 vectors; 384 dimensions
ONNX embedding	(1, 384), finite float32, normalized
RAG query checks	Relevant indexed passages returned
Vite production build	Completed successfully
GET /health via TestClient	HTTP 200; healthy response
Missing required fields	HTTP 422 as expected
Unsupported soil	HTTP 422 as expected
Unsupported water	HTTP 422 as expected
Unknown API route	HTTP 404 as expected

These are recorded local development checks, not proof that every external integration or live production scenario has been tested. Before submission, verify the deployed health endpoint and demonstrate one complete end-to-end plan if the service is available.

8.3 Screenshots to Add

• [Insert Screenshot 1 – Application home page]

• [Insert Screenshot 2 – Farm input form]

• [Insert Screenshot 3 – Crop recommendations and suitability]

• [Insert Screenshot 4 – Financial estimates]

• [Insert Screenshot 5 – RAG answer and supporting context]

• [Insert Screenshot 6 – Render dashboard showing Live status and /health response]

8.4 Sample Scenario

A development test scenario used Markapur, red soil, high water availability, Kharif season and 2 acres. Use actual application output for the final report; do not invent crop rankings or financial figures.

CHAPTER 9 – DEPLOYMENT

The project repository is hosted at https://github.com/kasi-vardhan/market-price-crop-planner. Render is the deployment target. The repository includes deployment configuration, a Dockerfile and a .python-version pin for the native Python build path. The production backend requirements are intentionally lean and use ONNX Runtime rather than loading the full sentence-transformers stack during API startup.

Deployment requires server-side environment variables for GROQ_API_KEY and DATA_GOV_API_KEY. The market resource URL is configurable. The project should be described as live only after the Render dashboard reports a successful deployment and the /health endpoint returns a healthy response. Add the verified public application URL and deployment screenshot before submission.

CHAPTER 10 – CONCLUSION AND FUTURE ENHANCEMENTS

10.1 Conclusion

The Market Price and Crop Planning Helper demonstrates how web technologies, rule-based agricultural logic, financial calculations, market-data integration and retrieval-augmented language assistance can be combined into a decision-support prototype. The system emphasizes transparent calculations, evidence retrieval and modular workflow orchestration. ONNX Runtime provides a lighter query-embedding inference path for deployment.

10.2 Future Enhancements

• Validate benchmark values with district-level agricultural experts.

• Add location-aware weather forecasts and soil-test integration.

• Expand historical mandi-price analytics and trend visualization.

• Include transport, storage, commission and post-harvest costs.

• Add Telugu, Tamil, Hindi and other regional-language support.

• Show source title, date and retrieved passage for grounded answers.

• Add saved farm profiles and downloadable crop plans.

• Add production monitoring, structured logs and automated end-to-end tests.

• Conduct field evaluation with farmers and agricultural extension professionals.

REFERENCES

1. Government of India, Open Government Data Platform India, data.gov.in – agricultural market-price resource and API documentation. https://www.data.gov.in/

2. Indian Council of Agricultural Research (ICAR), official publications and crop-production guidance. https://icar.org.in/

3. Acharya N. G. Ranga Agricultural University (ANGRAU), extension publications. https://angrau.ac.in/

4. FastAPI documentation. https://fastapi.tiangolo.com/

5. React documentation. https://react.dev/

6. LangGraph documentation. https://docs.langchain.com/oss/python/langgraph/overview

7. FAISS repository. https://github.com/facebookresearch/faiss

8. ONNX Runtime documentation. https://onnxruntime.ai/docs/

9. Hugging Face Tokenizers documentation. https://huggingface.co/docs/tokenizers/

10. Groq API documentation. https://console.groq.com/docs/overview

11. Render documentation. https://render.com/docs

Add exact titles, publication years, page numbers and access dates for the agricultural publications actually used to build the crop benchmark dataset and knowledge base.

APPENDIX – DETAILS TO CONFIRM BEFORE SUBMISSION

• Project team: ORBIT 431.

• Institution: Kalasalingam Academy of Research and Education.

• Department wording: confirm the official department name used by the institution.

• Academic year: 2026–2027.

• Guide name and designation: enter assigned guide details.

• Add student roll numbers if required.

• Add actual application URL only after verifying deployment.

• Replace screenshot placeholders with genuine application screenshots.

• Check the report against the Expert's prescribed template and correct any institutional formatting requirements.

This draft was prepared from the project details available in the conversation. The team must verify institutional details, source publications, screenshots, live deployment status and actual outputs before formal submission.